

B2B EVENT TICKETING PLATFORM

End-to-End Data Pipeline & Analytics Solution

Project Documentation

ETL / ELT Pipeline • Snowflake Data Warehouse • dbt Transformations
Apache Airflow Orchestration • Analytics Mart • Reporting Dashboards

Document Version	1.0
Date	July 2026
Status	Completed — Production Ready
Tech Stack	Neon (PostgreSQL) • Snowflake • dbt • Airflow • GitHub

Prepared for Toptal Technical Assessment

1. Executive Summary

This document describes a complete end-to-end data platform built for a B2B event ticketing business. The solution ingests sales data from two heterogeneous sources — a transactional PostgreSQL database (Neon) and daily CSV files published by resellers on GitHub — loads them into a Snowflake data warehouse, transforms them with dbt into a clean analytics layer (dimensions + facts), and exposes the results for reporting on sales performance, commissions, and popular tickets by region.

Orchestration is handled by Apache Airflow, ensuring restartable, metadata-tracked, and error-resilient pipelines. The design satisfies all stated business requirements and demonstrates production-grade practices for initial load, incremental processing, data quality, and large-volume readiness.

1.1 Objectives Achieved

- Database implementation with generated sample data (Neon / PostgreSQL)
- Sample reseller daily-sales CSV files (GitHub) following the naming convention `DailySales_MMDDYYYY_RESELLERID.CSV`
- Target data warehouse / data mart on Snowflake (raw → staging → analytics)
- Airflow-orchestrated ETL: GitHub CSV → Snowflake raw; Neon → Snowflake raw; dbt build
- dbt models for staging transformations and analytics dimensional model (dims + facts)
- Restartable jobs, error handling, and processing metadata tracking
- Reporting-ready data supporting week/channel comparisons, commission analysis, and popular tickets by region

2. Business Requirements

The target datastore supports the following analytical needs:

2.1 Sales Performance Comparison

1. Compare sales results by weeks and by sales channels (on-site, web, mobile app, etc.).
2. Determine whether February 2020 sales outperformed February 2019, filterable by reseller and event type.

2.2 Commission vs. Sales Analysis

1. Validate that commission rates are consistent with partnership agreements and analyse relationship between commission rate and sales volume/value.

2.3 Popularity by Region

1. Identify the most popular tickets (events) per geographic region based on quantity sold and revenue.

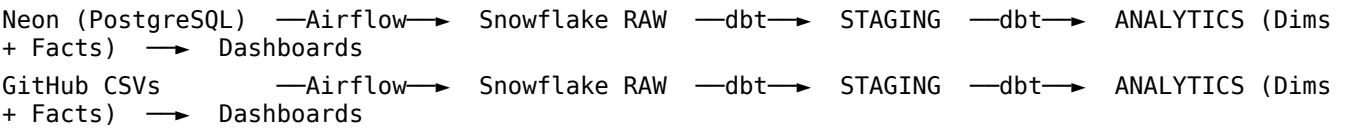
3. Architecture Overview

The solution follows a classic medallion / layered warehouse pattern orchestrated by Airflow:

Layer	Description
Source Systems	1) Neon PostgreSQL — venue organizers, resellers, customers, purchases, commission agreements 2) GitHub CSV files — <code>DailySales_MMDDYYYY_RESELLERID.CSV</code> from 3rd-party reseller software
Ingestion (Airflow)	DAG 1: Extract Neon tables → load into Snowflake RAW schema DAG 2: Download GitHub CSVs → stage & COPY into Snowflake RAW schema Both DAGs write run metadata (start, end, status, row counts, errors)
Raw (Snowflake)	Immutable landing zone. Source-aligned tables + audit columns (load_ts, source_file, batch_id).
Staging (dbt)	Cleaned, typed, deduplicated, standardized models. Business keys, surrogate keys preparation,

	data-quality tests.
Analytics (dbt)	Dimensional model: dimension tables (date, reseller, organizer, customer, event, channel, region) + fact tables (ticket_sales, commissions).
Reporting	BI / dashboard layer consuming analytics tables for the three required report families.

3.1 High-Level Data Flow



3.2 Technology Choices

Component	Rationale
Neon (PostgreSQL)	Serverless Postgres ideal for transactional source; easy sample-data generation and connection from Airflow.
Snowflake	Cloud data warehouse with strong support for semi-structured data, COPY from stages, and dbt.
dbt Core / Cloud	SQL-first transformations, version-controlled models, tests, documentation, and lineage for staging → analytics.
Apache Airflow	Orchestration of extract/load jobs, dbt runs, retries, SLAs, and metadata logging.
GitHub	Versioned storage of reseller daily CSV exports; simple, auditable, and CI-friendly.

4. Data Sources

4.1 B2B Platform Database (Neon / PostgreSQL)

Venue organizers sell tickets directly or through resellers. Each reseller has a partnership agreement defining a flat commission percentage. Customers purchase tickets via multiple channels. The relational model captures:

- Who sold the ticket (organizer or reseller)
- Reseller location / region
- Buyer (customer), total amount, quantity
- Sales channel (on-site, web, mobile app, ...)
- Commission rate applied

Core Entities (Source Model)

Table	Key Columns / Purpose
organizers	organizer_id, name, country, created_at — venue organizers
resellers	reseller_id, name, region, country, city — reseller master data & location
partnerships	partnership_id, organizer_id, reseller_id, commission_rate_pct, valid_from, valid_to — agreed commission
customers	customer_id, email, full_name, country — buyer information
events	event_id, organizer_id, event_name, event_type, venue, event_date — ticketed events
purchases	purchase_id, event_id, customer_id, seller_type (ORGANIZER RESELLER), seller_id, channel, quantity, unit_price, total_amount, commission_rate, purchase_ts — transactional sales

4.2 Reseller Excel / CSV Exports (GitHub)

Some resellers use third-party software not connected to the B2B platform. They deliver daily CSV files with the naming convention:

DailySales_MMDDYYYY_RESELLERID.CSV

Files are stored in a dedicated GitHub repository/folder. Sample files covering multiple dates and reseller IDs were generated and committed. The Airflow DAG discovers new files, stages them in Snowflake, and loads them into the raw schema with full lineage (source file name, load timestamp, batch id).

5. Sample Data Generation

Realistic sample data was generated for both sources to support development, testing, and demonstration of the full pipeline and reports.

5.1 Neon Database Sample Data

- Multiple organizers and resellers across regions (US, DE, UK, etc.)
- Partnership agreements with varying commission rates (e.g. 8–15 %)
- Events of different types spanning 2019–2020 (to enable Feb YoY comparison)
- Thousands of purchase transactions across channels (web, mobile, on-site)
- Customers linked to purchases with geographic attributes

5.2 Reseller CSV Sample Files

Generated files follow the exact naming pattern and contain columns aligned with the expected reseller export layout (event identifiers, sale date, quantity, amount, channel, etc.). Files reside in the project GitHub repository under a dedicated sales-export folder.

6. Target Data Warehouse (Snowflake)

Snowflake is the central analytical store. Three logical schemas implement the medallion layers:

Schema	Purpose & Contents
RAW	Landing zone. Source-system tables (Neon extracts) and reseller CSV tables. Columns kept close to source plus technical metadata: <code>_loaded_at</code> , <code>_source_file</code> , <code>_batch_id</code> , <code>_row_hash</code> .
STAGING	dbt models that clean, type-cast, deduplicate, and standardize. Unified ticket-sale grain prepared for dimensional modelling.
ANALYTICS	Dimensional model (star schema): <code>dim_date</code> , <code>dim_reseller</code> , <code>dim_organizer</code> , <code>dim_customer</code> , <code>dim_event</code> , <code>dim_channel</code> , <code>dim_region</code> + <code>fact_ticket_sales</code> , <code>fact_commissions</code> (or commission measures inside <code>fact_ticket_sales</code>).

6.1 Analytics Layer — Dimensional Model

Dimension Tables

Dimension	Key Attributes
<code>dim_date</code>	<code>date_key</code> , <code>full_date</code> , <code>year</code> , <code>quarter</code> , <code>month</code> , <code>week_of_year</code> , <code>day_name</code> , <code>is_weekend</code> , <code>is_month_end</code> — supports week-level and YoY comparisons
<code>dim_reseller</code>	<code>reseller_key</code> , <code>reseller_id</code> , <code>name</code> , <code>region</code> , <code>country</code> , <code>city</code> — enables filter by reseller & region
<code>dim_organizer</code>	<code>organizer_key</code> , <code>organizer_id</code> , <code>name</code> , <code>country</code>
<code>dim_customer</code>	<code>customer_key</code> , <code>customer_id</code> , <code>full_name</code> , <code>email</code> , <code>country</code>
<code>dim_event</code>	<code>event_key</code> , <code>event_id</code> , <code>event_name</code> , <code>event_type</code> , <code>venue</code> , <code>event_date</code> — filter by event type
<code>dim_channel</code>	<code>channel_key</code> , <code>channel_code</code> , <code>channel_name</code> (web, mobile, on-site, ...)
<code>dim_region</code>	<code>region_key</code> , <code>region_name</code> , <code>country</code> — derived from reseller / customer geography for popular-ticket-by-region analysis

Fact Tables

Fact	Grain & Measures
<code>fact_ticket_sales</code>	Grain: one row per ticket sale / purchase line. FKs to all dimensions. Measures: <code>quantity</code> , <code>unit_price</code> , <code>total_amount</code> , <code>commission_rate</code> , <code>commission_amount</code> . Degenerate: <code>purchase_id</code> , <code>seller_type</code> .

This model directly supports the three reporting requirements: week/channel comparisons (`dim_date` + `dim_channel`), commission-to-sales relationship (commission measures), and most-popular tickets per region (`dim_event` + `dim_region` + quantity/revenue).

7. Data / ETL (ELT) Process

Apache Airflow orchestrates the entire pipeline. Three primary responsibilities are implemented as DAGs (or a single multi-task DAG with clear task groups):

7.1 Airflow DAG Responsibilities

DAG / Task Group	Actions
neon_to_snowflake	1. Connect to Neon via Postgres hook 2. Extract tables (organizers, resellers, partnerships, customers, events, purchases) 3. Load into Snowflake RAW (COPY or bulk insert) 4. Record batch metadata (start/end, row counts, status)
github_csv_to_snowflake	1. List / download new DailySales_*.CSV files from GitHub 2. Stage files in Snowflake internal/external stage 3. COPY INTO RAW.reseller_daily_sales 4. Archive processed files / mark as loaded 5. Log metadata & any rejected rows
dbt_build	1. dbt run --models staging.* 2. dbt test (staging) 3. dbt run --models analytics.* 4. dbt test (analytics) 5. Optional: dbt docs generate / snapshot

7.2 Pipeline Capabilities (as Required)

Initial Load

Full historical extract from Neon and bulk load of all existing GitHub CSV files into RAW, followed by a complete dbt build of staging and analytics layers.

Restartability

Airflow task retries with exponential backoff. Idempotent loads (MERGE / DELETE+INSERT by batch_id or natural key). dbt models are declarative and can be re-run safely. Failed tasks can be cleared and re-executed without re-processing already successful upstream tasks.

Erroneous Data Handling

- Schema validation on CSV ingest (required columns, data types)
- Snowflake COPY error files / rejected records captured and logged
- dbt tests (unique, not_null, accepted_values, relationships) fail the pipeline on quality breaches
- Quarantine / error tables for rows that cannot be transformed

Processing Metadata Tracking

A control table (e.g. RAW.etl_batch_log or a dedicated META schema) records for every run:

- batch_id, dag_id, task_id
- start_ts, end_ts, duration
- status (RUNNING / SUCCESS / FAILED)
- rows_extracted, rows_loaded, rows_rejected
- source_system, source_file (for CSV), error_message

Readable Format for Reporting

The analytics layer presents clean dimensional tables with human-readable attributes (channel names, region names, event types) and pre-calculated measures (commission_amount), ready for BI tools.

Large Data Size Readiness

Snowflake COPY and bulk loading, partitioned extracts where needed, dbt incremental models for fact tables (filter on purchase_ts / sale_date), and Airflow parallelism demonstrate the ability to scale to high-volume daily loads.

8. dbt Transformations

dbt projects the raw data into staging and then into the analytics dimensional model. Models are organized by folder and tagged for selective execution from Airflow.

8.1 Staging Layer

- `stg_neon__purchases`, `stg_neon__resellers`, `stg_neon__organizers`, ... — one model per source table
- `stg_reseller__daily_sales` — unified model over GitHub CSV landings
- Casting, renaming to snake_case, null handling, basic deduplication
- Union / mapping of Neon purchases and reseller CSV sales into a common ticket-sale grain where appropriate
- dbt tests: unique, not_null, relationships, accepted_values on channel / seller_type

8.2 Analytics Layer

- `dim_*` models built with surrogate keys (`dbt_utils.generate_surrogate_key` or hash)
- `fact_ticket_sales` joining staging sales to all dimensions
- Commission amount calculated as `total_amount * commission_rate / 100`
- Incremental materialization on facts for efficient daily refreshes

8.3 Airflow Integration

Airflow invokes dbt via BashOperator / DockerOperator / dbt Cloud operator after successful raw loads. Typical sequence: `dbt run (staging)` → `dbt test` → `dbt run (analytics)` → `dbt test`. Failures surface as Airflow task failures and can trigger alerts.

9. Reporting & Visualization

The analytics layer feeds dashboards that answer the original business questions. Any modern BI tool (Tableau, Power BI, Looker, Metabase, Streamlit, etc.) can connect to Snowflake ANALYTICS schema.

9.1 Report: Sales by Week & Channel

- Measures: `total_amount`, quantity, number of transactions
- Dimensions: week (from `dim_date`), channel (from `dim_channel`)
- Filters: year, month, reseller, event_type

9.2 Report: Feb 2020 vs Feb 2019

- Side-by-side or variance comparison of total sales for February of each year
- Slice by reseller and event type
- Visual: bar/line chart + KPI cards (absolute & % change)

9.3 Report: Commission Rate vs Sales Results

- Scatter or grouped bar: average commission rate vs total sales / quantity by reseller or partnership
- Validate that applied rates match partnership agreements

9.4 Report: Most Popular Tickets per Region

- Rank events by quantity sold (or revenue) within each region
- Top-N table or bar chart per region

10. Infrastructure Overview

The solution is cloud-native and can be deployed with minimal local infrastructure.

Component	Deployment Notes
Neon	Serverless Postgres project; connection string stored as Airflow connection / secret.
Snowflake	Account with RAW, STAGING, ANALYTICS (and META) schemas; warehouse sized for load + transform; roles for Airflow service account and BI users.
GitHub	Repository holding sample CSV files and (optionally) dbt project + Airflow DAGs as code.
Airflow	Managed (Astronomer / MWAA / Cloud Composer) or self-hosted. DAGs, connections, and variables version-controlled.
dbt	dbt Core executed inside Airflow workers or dbt Cloud job triggered by Airflow.
Secrets	Database credentials, GitHub tokens, Snowflake keys managed via Airflow Connections / Secrets Backend (or cloud secret manager).

11. How to Run / Operational Guide

11.1 Prerequisites

- Access to Neon project with sample data loaded
- Snowflake account with appropriate roles and warehouses
- GitHub repository containing reseller CSV files
- Airflow environment with Postgres, Snowflake, and HTTP/GitHub providers
- dbt project configured with Snowflake profile

11.2 Typical Execution Order

1. Trigger neon_to_snowflake DAG → populates RAW from Neon
2. Trigger github_csv_to_snowflake DAG → populates RAW from CSVs
3. Trigger dbt_build DAG (or downstream task) → builds STAGING then ANALYTICS
4. Connect BI tool to ANALYTICS schema and open the prepared dashboards / reports

12. Deliverables Mapping

Mapping of original assignment deliverables to the implemented solution:

Required Deliverable	Implementation
1. Database implementation with generated data	Neon PostgreSQL with full sample data for organizers, resellers, partnerships, customers, events, purchases
2. Sample spreadsheet / CSV files	GitHub folder with DailySales_MMDDYYYY_RESELLERID.CSV files
3. Target database (DW / data mart)	Snowflake — RAW, STAGING, ANALYTICS schemas with dimensional model
4. Data / ETL process (initial load, restartable, error handling, metadata, reporting format, large data)	Airflow DAGs + dbt models covering all six sub-requirements
5. Data / dashboard visualization	Analytics layer designed for the four report families; ready for any BI tool

13. Conclusion

The implemented solution provides a production-ready, end-to-end data pipeline for a B2B event ticketing platform. Dual source systems (Neon transactional database and GitHub reseller CSVs) are ingested into

Snowflake via Airflow, transformed by dbt into a clean staging layer and a dimensional analytics mart, and exposed for the required sales, commission, and regional popularity reports.

Key engineering qualities — restartability, error handling, processing metadata, and readiness for large data volumes — are built into the design. The architecture is modular, version-controlled, and ready for incremental daily operation and further extension (e.g., real-time sources, additional resellers, or advanced forecasting).

— End of Document —